

Mini-project 2: Weather vs. Climate

The goals of this project are to:

- Develop an overview of the local climate and its “normal variation.”
- Understand how local weather varies compared to climate.
- Use Python to visualize measured data graphically.
- Evaluate your methods for effective data visualization.
- Gain experience with teamwork and collaboration.

Data sources for Norway: <https://www.met.no/frie-meteorologiske-data/frie-meteorologiske-data>.

The easy way in is to use the web interface: <https://seklima.met.no/>

For other nations, you could search for national sources of historical weather information.

Time use This project is designed to take about 6–8 hours in total.

Work sessions: Wednesday, September 16 and Wednesday, September 23, 14:15–16:00.

Submission deadline: **September 29**.

Part 1. Local Temperature

Create a chart based on historical data that helps you answer the question: “**Is this temperature normal for X?**” (where *X* is your hometown or the place you grew up).

Tasks:

1. Create a **climate profile** of temperature.
2. Plot the maximum and minimum temperatures for 2025 and compare them with a reference set of historical measurements (e.g., a 30-year climate normal period such as 1991–2020, if your data source provides one).

Teamwork tasks:

- Ensure that everyone understands the task.
- Develop and share the algorithm for generating the charts.
- Discuss whether your way of presenting the data is clear and meaningful.
- Reflect on whether your team is working efficiently.

Deliverables (Part 1):

1. Charts with a short description in the text.
2. Source code for plotting you add in an Appendix A.
3. Team reflections on “Is this an efficient way to work for us?”.
4. Documentation of AI use, prompts, and requests.

Part 2. Local Precipitation

Create a chart based on historical data that helps you answer the question:

“Is this amount of rain normal for X?”

Tasks:

- Identify the normal monthly precipitation.
- Determine which months receive the most precipitation.
- Investigate whether some months are statistically or climatologically drier.
- Examine whether precipitation has increased over time.

Teamwork tasks:

- Define what you mean by “amount of precipitation” (are there different options?).
- Explore whether the measurements can be fitted to a statistical distribution.

Deliverables (Part 2):

1. Charts with comments in the text.
2. A discussion of how you defined “amount of precipitation” and the options considered.
3. A way of presenting the data that can be compared with other groups.
4. Source code for plotting (Appendix B).
5. Documentation of AI use, prompts, and requests.

Part 3. Pressure or Wind Data (Choose one)

a) Local Pressure

Create a chart based on historical data that helps you answer the question:

“Is this pressure normal for location X?”

Tasks:

- Explore variations in pressure during a month.
- Investigate whether pressure has changed over time.

b) Local Wind

Create a chart based on historical data that helps you answer the question:

“Is this wind normal for X?”

Tasks:

- Identify normal monthly wind patterns.
- Determine which months are windier.
- Explore whether some months are statistically or climatologically calmer.

(Note: Wind data is more complex than pressure data due to higher frequency variation and directional components.)

Deliverables (Part 3):

1. Charts with comments in the text.
2. Clear presentation of the visualized data.
3. A way of presenting the data that can be compared with other groups.
4. Source code for plotting (Appendix C).
5. Documentation of AI use, prompts, and requests.

Project Deliverables

Each student must submit their own individual report in Blackboard. The report must include:

- Charts and discussion for each part.
- Source code (Your **Appendices A–C**).
- Completed **AI Declaration Form** (Your **Appendix D**).
- Documentation of AI use (Your **Appendix E**).
- Reflections on teamwork and efficiency (Your **Appendix F**).

Use of AI

You are encouraged to use AI for:

- Developing plotting algorithms.
- Exploring statistical models (e.g., precipitation distributions).

Important:

You must fill in the official AI declaration form (Your **Appendix D**) and attach it to your report. All AI use, prompts, and requests must be documented in your **Appendix E**.

Appendix A Sample code frost API

In order to use the API directly from <https://frost.met.no/howto.html>. Once you have your private key you can run this demo script to download data from

```
import requests
import pandas as pd
from datetime import datetime, timezone
from time import sleep
import matplotlib.pyplot as plt
# -----
# Settings
# -----

client_id = 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx'

STATION = "SN68860"
START_YEAR = 2000
END_YEAR = 2026

url = "https://frost.met.no/observations/v0.jsonld"

all_data = []

for year in range(START_YEAR, END_YEAR + 1):
    print(f"{year}: ", end="")
    params = {
        "sources": STATION,
        "elements": "air_temperature",
        "referencetime": f"{year}-01-01/{year+1}-01-01",
        "timeresolutions": "PT1H",
        "timeoffsets": "default",
        "levels": "default",
        "qualities": "0,1,2,3,4"
    }
    r = requests.get(
        url,
        params=params,
        auth=(client_id, "")
    )
    if r.status_code == 404:
        print("no data")
        continue
    if r.status_code != 200:
        print("error", r.status_code)
        print(r.text[:300])
```

```

        continue
    data = r.json()
    n = 0
    # pick the time and temperature data
    for item in data.get("data", []):
        for obs in item["observations"]:
            all_data.append({
                "time": item["referenceTime"],
                "temperature": obs["value"]
            })
        n += 1
    print(f"{n:,} observations")
    sleep(0.2)

# -----
# Combine all observations
# -----

df = pd.DataFrame(all_data)
df["time"] = pd.to_datetime(
    df["time"],
    utc=True
)
df = (
    df
    .sort_values("time")
    .drop_duplicates(subset="time")
    .set_index("time")
)
# Now the dataframe contains two columns time and temperature
# -----
# Calculate hourly means
# -----

df["daynumber"] = (
    df.index.dayofyear
    + df.index.hour / 24
)

# -----

```

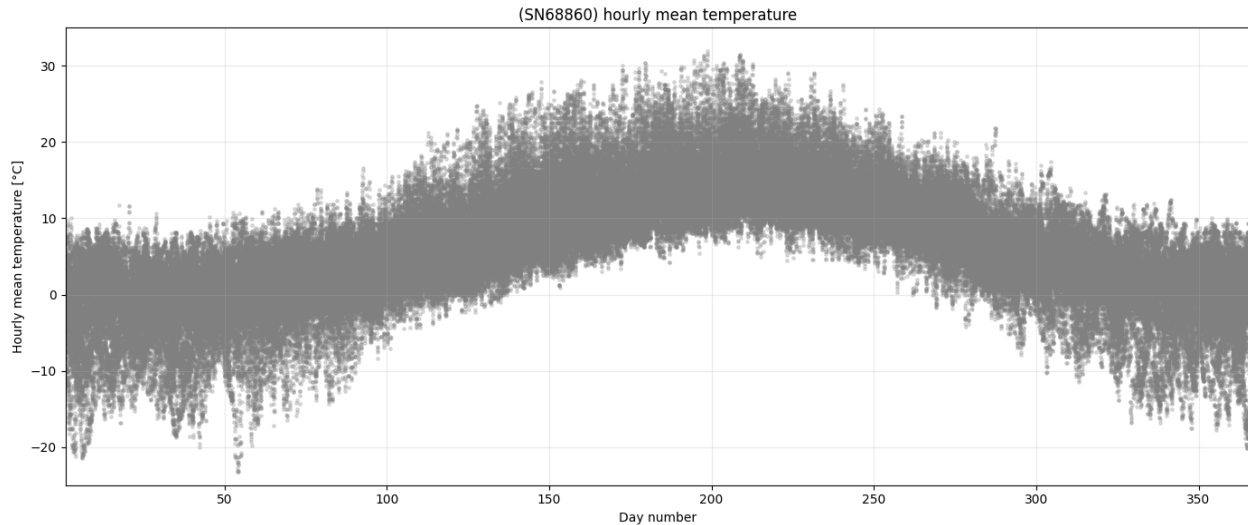
```

# Save
# Once saved you don't have to reload the data
# -----

df.to_csv(
    f"{STATION}_hourly_mean_temperature.csv"
)
# -----
# Plot scatter diagram
# -----

fig, ax = plt.subplots(figsize=(14, 6))
ax.scatter(
    df["daynumber"],
    df["temperature"],
    s=6,
    color="gray",
    alpha=0.25
)
ax.set_xlabel("Day number")
ax.set_ylabel("Hourly mean temperature [°C]")
ax.set_xlim(1, 366)
ax.set_ylim(-25, 35)
ax.set_title(
    f"({STATION}) hourly mean temperature"
)
ax.grid(alpha=0.25)
fig.tight_layout()
plt.show()

```



Plotting the mean temperature as a function of daynumber with standard deviation and a max and min range.

```
temperature_mean=np.zeros(365)
temperature_std=np.zeros(365)
temperature_max=np.zeros(365)
temperature_min=np.zeros(365)
daynumber=np.zeros(365)
for d in range(365):
    temperature_mean[d]=np.mean(df[(df["daynumber"] >= d) & (df["daynumber"] <
d+1)]["temperature"])
    temperature_std[d]=np.std(df[(df["daynumber"] >= d) & (df["daynumber"] <
d+1)]["temperature"])
    temperature_min[d]=np.min(df[(df["daynumber"] >= d) & (df["daynumber"] <
d+1)]["temperature"])
    temperature_max[d]=np.max(df[(df["daynumber"] >= d) & (df["daynumber"] <
d+1)]["temperature"])
    daynumber[d]=d
```

```
fig, ax = plt.subplots(figsize=(14, 7))
# All daily means from all years
ax.scatter(
    daynumber,
    temperature_mean,
    s=6,
    color="gray",
    alpha=0.5,
    label="Daily mean temperature"
)
plt.errorbar(
```

```

    daynumber,
    temperature_mean,
    yerr=temperature_std,
    fmt='o',
    capsize=3
)
# 95% range of daily temperatures
ax.fill_between(
    daynumber,
    temperature_min,
    temperature_max,
    alpha=0.25,
    label="100% range"
)

ax.set_xlabel("Day number")
ax.set_ylabel("Daily mean temperature [°C]")
ax.set_xlim(1, 366)
ax.set_title(
    f"({STATION})\n"
    "Daily mean temperature and climatological 95% range"
)
ax.legend()
ax.grid(alpha=0.2)
fig.tight_layout()
plt.show()

```

